

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
 ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
 «САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕР-
 СИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
 МЕХАНИКИ И ОПТИКИ»

РАБОЧАЯ ПРОГРАММА ДИСЦИПЛИНЫ
Программная инфраструктура и управление разработкой

Направления подготовки	09.03.01 Информатика и вычислительная техника 09.03.04 Программная инженерия 10.03.01 Информационная безопасность 44.03.04 Профессиональное обучение
Квалификация выпускника	БАКАЛАВР
Образовательные программы	Компьютерные системы и технологии Системное и прикладное программное обеспечение Нейротехнологии и программирование Технологии защиты информации Компьютерные технологии в дизайне
Форма обучения	очная

Трудоемкость		Се- местр	Вид контроля (экз./диф.зач./з ач.)	Контакт- ная ра- бота, час.	Занятий лекц. типа, час.	Лаборат. занятий, час.	Практич. занятий, час.	СРО, час.
зач. ед.	час.							
4	144	6	экзамен	52.8	16	16	0	91.2

Санкт-Петербург
2026г.

РАБОЧАЯ ПРОГРАММА ДИСЦИПЛИНЫ

Разработана:

структурное подразделение: факультет ПИ и КТ

_____ Миху В.Д.

_____ Цопа Е.А.

1. ЦЕЛИ ОСВОЕНИЯ ДИСЦИПЛИНЫ

Целью освоения дисциплины является достижение следующих результатов обучения:

Код компетенции	Код индикатора(ов) достижения компетенций	Планируемые результаты обучения
09.03.01 ОПК-5	ОПК-5.1 Инсталляция и настройка ПО, контейнерных платформ и создание воспроизводимых сред.	Знание: установка и настройка ПО, основы контейнеризации, требования к воспроизводимым средам. Умение: создавать среды запуска приложений, применять контейнеризацию. Навыки: формирование переносимых окружений и работа с автоматизированными проектами.
09.03.01 ОПК-7	ОПК-7.1 Настройка программно-аппаратных комплексов и аудит инфраструктуры.	Знание: структура ПАК, методы анализа инфраструктуры. Умение: выявлять и устранять инфраструктурные ограничения. Навыки: проведение аудита и подготовка улучшений.
09.03.02 ОПК-7	ОПК-7.1: Автоматизация, CI/CD, оркестрация и конфигурационное управление в жизненном цикле ПО.	Знание: инструменты автоматизации, CI/CD, оркестрация, конфигурационное управление, критерии выбора технологий. Умение: выбирать и применять инструменты автоматизации, масштабировать и поддерживать среды исполнения. Навыки: настройка CI/CD, оркестрации и автоматизации администрирования.
09.03.03 ОПК-3	ОПК-3.1 Мониторинг состояния систем и оценка надежности сервисов с учетом ИБ.	Знание: анализ состояния систем, основы информационной безопасности, оценка надежности. Умение: выявлять отклонения и анализировать состояние систем. Навыки: мониторинг и оценка устойчивости сервисов.
09.03.03 ОПК-8	ОПК-8.1 Работа с метриками, ролями, коммуникацией и управлением изменениями.	Знание: метрики процессов, командные роли, принципы коммуникации и управления изменениями. Умение: анализировать показатели, распределять задачи, выстраивать взаимодействие. Навыки: командная работа, согласование изменений и оценка эффективности процессов.

09.03.04 ОПК-2	ОПК-2.1 Применение DevOps-практик и CI/CD в разработке ПО.	Знание: DevOps-культура, CI/CD, практики командной разработки. Умение: применять DevOps-подходы в организации разработки. Навыки: использование практик совместной разработки.
09.03.04 ОПК-6	ОПК-6.1 Разработка, тестирование, отладка и анализ логов программных систем.	Знание: основы разработки и тестирования, работа с логами и технической документацией. Умение: разрабатывать, тестировать, отлаживать и анализировать поведение систем. Навыки: работа с автоматизированными проектами, анализ логов, улучшение надежности систем.

2. СТРУКТУРА И СОДЕРЖАНИЕ ДИСЦИПЛИНЫ

№ раздела	Наименование раздела дисциплины	Распределение часов по дисциплине, часы					
		Контактная работа	Занятия лекционного типа	Лабораторные занятия	Практические занятия	СРО	Всего часов
1	Основы DevOps и управление разработкой	13,2	4	4	0	10,8	24
2	Контейнеризация и Docker	13,2	4	4	0	26,8	40
3	Оркестрация и автоматизация	13,2	4	4	0	26,8	40
4	Наблюдаемость, безопасность и эксплуатация	13,2	4	4	0	26,8	40
ИТОГО:		52,8	16	16	0	91,2	144

Дисциплина реализуется:

- без использования онлайн-курса ☒

Номер (а) из перечня онлайн-курсов, указанных в учебно-методическом обеспечении дисциплины

- с использованием онлайн-курсов (смешанное обучение)

- может быть заменена онлайн-курсом

№ раз-дела	Наименование раз-дела дисциплины	Содержание
1	Основы DevOps и управление разработкой	• Предпосылки появления DevOps, цели, ценности и культурные аспекты взаимодействия команд разработки и эксплуатации. • Принципы автоматизации, непрерывной интеграции и доставки. Жизненный цикл разработки программного обеспечения (SDLC). • Модели процессов разработки, области применения. • Организация командной работы, управление задачами и проектами, системы отслеживания задач, практики гибкой разработки. • Системы контроля версий, Git, стратегии ветвления, роль в совместной разработке и CI/CD.
2	Контейнеризация и Docker	• Принципы контейнеризации, отличие от виртуализации. • Архитектура Docker, основные компоненты: образы, контейнеры, сети, тома. Dockerfile, слоистая структура образов, оптимизация, снижение размера, базовые аспекты безопасности. • Управление контейнерами, сетевые режимы, хранение данных. Использование Docker в CI/CD: сборка, тестирование, развертывание приложений.
3	Оркестрация и автоматизация	• Оркестрация контейнеризированных приложений, Kubernetes, архитектура и ключевые объекты. • Развертывание приложений, масштабирование, обновления, управление конфигурациями. Helm, шаблонизация и управление пакетами. • Инфраструктура как код (IaC): Terraform, управление ресурсами; Ansible, конфигурационное управление и автоматизация. • CI/CD пайплайны, структура, инструменты, этапы автоматизации.
4	Наблюдаемость, безопасность и эксплуатация	• Наблюдаемость систем: метрики, логи, трассировки. Инструменты мониторинга, визуализация, алертинг. • DevSecOps, безопасность в жизненном цикле разработки, анализ уязвимостей, управление доступами и секретами. Конфигурации и среды, управление параметрами. • Эксплуатация и SRE: SLA, SLO, SLI, инциденты, отказоустойчивость систем.

3. УЧЕБНО-МЕТОДИЧЕСКОЕ ОБЕСПЕЧЕНИЕ ДИСЦИПЛИНЫ

Литература:

- 1) Гош С. “Docker без секретов”: Пер. с англ. — СПб.: БХВ Петербург, 2023. — 224 с.: ил. ISBN 978-5-9775-1196-4
- 2) Уилсон К. “Трокаем Continuous Delivery” — СПб.: Питер, 2024. — 400 с.: ил. — (Серия “Библиотека программиста”). ISBN 978-5-4461-2372-8
- 3) Джеймс С. Валлери Л. “Kubernetes и сети. Многоуровневый подход” — СПб.: БХВ Петербург, 2024. — 320 с.: ил. ISBN 978-5-9775-1855-0
- 4) П. Коннинг “Инструментарий agile-лидера. Научитесь преуспевать с помощью самоуправляемых команд” — СПб.: БХВ Петербург, 2021. — 224 с.: ил. ISBN 978-5-9775-6721-3
- 5) Джин К., Патрик Д., Джон У., Джез Х. “Руководство по DevOps” — СПб.: Манн, Иванов и Фербер, Петербург, 2018. — 512 с.: ил. ISBN 978-5-00100-750-0

4. ОЦЕНОЧНЫЕ СРЕДСТВА ДЛЯ ПРОВЕДЕНИЯ ПРОМЕЖУТОЧНОЙ АТТЕСТАЦИИ ПО ДИСЦИПЛИНЕ

Порядок оценки освоения обучающимися учебного материала определяется содержанием следующих разделов дисциплины:

№ п/п	Наименование раздела дисциплины (модуля)	Компетенция(и)	Индикатор(ы) достижения компетенции	Оценочные средства текущего контроля успеваемости
1.	Основы DevOps и управление разработкой	ОПК-2, ОПК-8, ОПК-9	ОПК-2.1, ОПК-8.1, ОПК-9.1	Текущие контрольные задания. Защита лабораторных работ. Рубежные контрольные работы.
2.	Контейнеризация и Docker	ОПК-5, ОПК-7	ОПК-5.1, ОПК-7.1	Текущие контрольные задания. Защита лабораторных работ. Рубежные контрольные работы.
3.	Оркестрация и автоматизация	ОПК-7	ОПК-7.1, ОПК-7.2	Текущие контрольные задания. Защита лабораторных работ. Рубежные контрольные работы.
4.	Наблюдаемость, безопасность и эксплуатация	ОПК-3, ОПК-6, ОПК-8	ОПК-3.1, ОПК-6.1, ОПК-8.1	Текущие контрольные задания. Защита лабораторных работ. Рубежные контрольные работы.

Таблица планирования оценивания результатов обучения по дисциплине (3 семестр):

	Модуль 5										Модуль 6										Промежуточная аттестация по дисциплине	
	Текущий контроль по точкам								Рубежный контроль		Текущий контроль по точкам								Рубежный контроль			
	1		2		3		4				1		2		3		4					
	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max	[min]	max
Защита отчета по л.р.			6	9,5			6	9,5					6	9,5			6	9,5				
Выполнение контрольных работ	1	2	1	2	1	2	1	2			1	2			1	2						
Рубежное тестирование									6	10									6	10		
Экзамен																				12	20	
Личностные качества							3	5									3	5				
Балловая стоимость одной точки	1	2	7	11,5	1	2	10	16,5	6	10	1	2	6	9,5	1	2	9	14,5	6	10	12	20
Накопление баллов	1	2	8	13,5	9	15,5	19	32	25	42	1	2	7	11,5	8	13,5	17	28	23	38		
Итого:																					60	100

ТИПОВЫЕ КОНТРОЛЬНЫЕ ЗАДАНИЯ ИЛИ ИНЫЕ МАТЕРИАЛЫ, НЕОБХОДИМЫЕ ДЛЯ ОЦЕНКИ ДОСТИЖЕНИЯ ЗАПЛАНИРОВАННЫХ РЕЗУЛЬТАТОВ ОБУЧЕНИЯ

Оценивание уровня учебных достижений обучающихся по дисциплине осуществляется в виде текущего и промежуточного контроля.

Текущий контроль успеваемости по дисциплине осуществляется в форме (формах) выполнения и защиты лабораторных работ, написания текущих контрольных работ, выполнения рубежных контрольных тестов.

Текущие контрольные работы

Для всех тестов происходит пересчет рейтинга теста в баллы по следующим критериям:

- рейтинг теста меньше 60% – 0 баллов,
- рейтинг теста 60% – min балл,
- рейтинг теста 100% – max балл,
- рейтинг теста от 50% до 100% – пересчет по формуле:
$$([\text{рейтинг теста}] - 50) / 50 * ([\text{max балл}] - [\text{min балл}]) + [\text{min балл}]$$

Пример вопросов текущей контрольной работы:

1. Перечислите этапы жизненного цикла разработки ПО (SDLC) и опишите их основные особенности.
2. Что такое система контроля версий (VCS)? Приведите примеры и опишите основные операции: commit, branch, merge.
3. Что такое тома (volumes) в Docker? Зачем нужны тома и как они используются для сохранения данных за пределами жизненного цикла контейнера?

ЛАБОРАТОРНЫЕ РАБОТЫ

№ п/п	Номер раздела дисциплины	Наименование лабораторной работы	Трудоемкость, часов
1	1	Работа с контейнерами	8
2	2	Работа с кластерами	8
3	3	Пайплайны, автоматизация	8
4	4	Инфраструктура как код	8
Итого:			32

ПРИМЕР ЗАДАНИЯ НА ЛАБОРАТОРНУЮ №1

1. Цель работы

Закрепить теоретические знания по темам лекций 1–4 раздела «Основы DevOps и управление разработкой»: получить практический опыт работы с Docker, Git и рабочими процессами разработки на примере реального многокомпонентного приложения.

Студент должен уметь:

- клонировать репозиторий и работать с историей изменений в Git;
- запускать многоконтейнерное приложение с помощью Docker Compose;
- анализировать образы и контейнеры Docker;
- вносить изменения в Dockerfile и собирать пользовательский образ;
- понимать принципы DevOps и жизненного цикла разработки ПО;
- строить технические диаграммы, описывающие архитектуру и процессы.

2. Варианты заданий

Каждый студент выполняет работу согласно своему варианту. Вариант определяет контейнер для детального исследования (задание 2) и URL-источник для встраиваемого iframe (задание 3).

Вариант	Контейнер для inspect	iframe-источник для страницы vote
1	vote	https://ru.wikipedia.org/wiki/Docker
2	result	https://docs.docker.com/get-started/
3	worker	https://github.com/docker-samples/example-voting-app
4	db	https://hub.docker.com/
5	redis	https://12factor.net/
6	vote	https://semver.org/lang/ru/
7	result	https://keepachangelog.com/ru/1.0.0/
8	worker	https://www.conventionalcommits.org/ru/v1.0.0/
9	db	https://docs.github.com/en/actions
10	redis	https://about.gitlab.com/topics/ci-cd/
11	vote	https://trello.com/home
12	result	https://jira.atlassian.com/
13	worker	https://www.atlassian.com/agile/scrum
14	db	https://www.atlassian.com/agile/kanban
15	redis	https://agilemanifesto.org/iso/ru/manifesto.html
16	vote	https://git-scm.com/doc
17	result	https://www.gitflow.com/
18	worker	https://nvie.com/posts/a-successful-git-branching-model/
19	db	https://docs.docker.com/compose/
20	redis	https://prometheus.io/docs/introduction/overview/
21	vote	https://grafana.com/docs/grafana/latest/
22	result	https://kubernetes.io/ru/docs/home/
23	worker	https://www.jenkins.io/doc/
24	db	https://docs.ansible.com/
25	redis	https://www.terraform.io/intro
26	vote	https://www.linux.org/
27	result	https://nginx.org/ru/docs/
28	worker	https://flask.palletsprojects.com/en/stable/

29	db	https://www.postgresql.org/docs/
30	redis	https://redis.io/docs/

Примечание. Если URL не открывается в iframe из-за политики X-Frame-Options, допускается замена на аналогичный ресурс — факт замены и причину необходимо зафиксировать в отчёте.

3. Подготовка рабочего окружения

3.1. Клонирование репозитория

Выполните клонирование репозитория example-voting-app в рабочий каталог:

```
git clone https://github.com/dockeramples/example-voting-app.git
cd example-voting-app
```

Исследуйте структуру репозитория и историю коммитов:

```
git log --oneline --graph --decorate --all | head -30
git branch -a
git tag
```

Зафиксируйте в отчёте: количество веток, тегов, авторов коммитов, дату последнего коммита. Опишите структуру каталогов (что находится в каждой директории).

3.2. Анализ docker-compose файлов

В репозитории присутствуют несколько Compose-файлов. Изучите их:

```
ls *.yaml *.yml
cat docker-compose.yml
```

Ответьте в отчёте на вопросы:

- Сколько сервисов описано в docker-compose.yml? Перечислите их.
- Какие порты проброшены на хост? Для каких сервисов?
- Какие зависимости между сервисами (depends_on)?
- Какие переменные окружения заданы? Для чего они нужны?
- Какие тома (volumes) используются и для чего?
- Для каких сервисов используется готовый образ (image:), а для каких — сборка из Dockerfile (build:)?

4. Задание 1: Запуск и исследование приложения

4.1. Запуск с помощью Docker Compose

Запустите приложение в фоновом режиме:

```
docker compose up -d
```

Дождитесь запуска всех контейнеров. Проверьте статус:

```
docker ps
docker compose ps
```

Скопируйте вывод обеих команд в отчёт. Обратите внимание на различие в отображаемой информации.

```
docker compose logs --tail=20
```

4.2. Исследование работоспособности приложения

Откройте в браузере интерфейс голосования (порт 5000) и интерфейс результатов (порт 5001). Проголосуйте за один из вариантов и убедитесь, что результат отображается на второй странице.

Для проверки сетевого взаимодействия выполните:

```
docker compose exec vote curl -s http://redis:6379 || true
docker compose exec worker env | sort
docker network ls
docker network inspect example-voting-app_front-tier
```

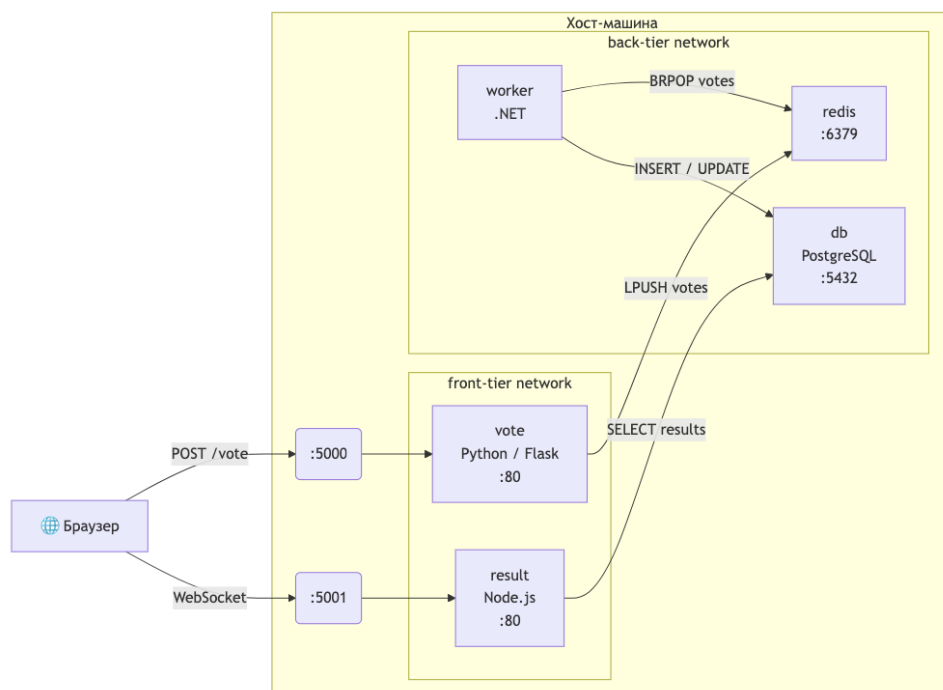
В отчёте:

- Приведите скриншот страницы голосования и страницы результатов после голосования.
- Сколько сетей создано? Какие контейнеры в какие сети входят?
- Опишите схему взаимодействия сервисов: как голос проходит путь от браузера до базы данных.

Диаграмма 1 — архитектура и поток данных приложения

Постройте диаграмму потока данных в приложении. Диаграмма должна отражать все сервисы, сети (front-tier, back-tier) и направления потоков данных. Включите в отчёт как исходный код диаграммы, так и её рендер (скриншот или экспорт из mermaid.live).

Ориентировочная структура — доработайте и скорректируйте по реальным данным из `docker network inspect` и `docker compose ps`:



Вопросы к диаграмме (ответьте в отчёте):

- Какие сервисы входят только во `front-tier`, какие — только в `back-tier`, а какие — в обе сети? Почему именно так разделены?
- Что произойдёт, если контейнер `redis` упадёт? Какие сервисы это затронет и почему?
- Как изменится диаграмма, если добавить балансировщик нагрузки перед сервисом `vote`?

4.3. Наблюдение за метриками во время работы

Запустите мониторинг ресурсов и сделайте скриншот:

```
docker stats --no-stream
```

Нагрузите приложение — несколько раз проголосуйте. Запустите `stats` повторно:

```
docker stats --no-stream
```

Сравните показатели CPU, памяти и сетевой активности до и после нагрузки. Какой контейнер потребляет больше всего памяти? CPU?

5. Задание 2: Детальное исследование контейнера (по варианту)

5.1. Сбор информации через `docker inspect`

Исследуйте контейнер, соответствующий вашему варианту:

```
docker inspect <имя_контейнера>
```

Извлеките и зафиксируйте в отчёте следующие сведения (используйте фильтрацию через `--format` или `jq`):

1. Идентификаторы: полный ID контейнера, короткий ID (первые 12 символов), Image ID.
2. Сетевые параметры: IP-адрес, MAC-адрес, сеть(и), шлюз, маска подсети.
3. Переменные окружения (Environment) — полный список.
4. Точка входа (Entrypoint) и команда запуска (Cmd).
5. Смонтированные тома и биндинги (Mounts).
6. Рабочая директория (WorkingDir) внутри контейнера.
7. Время создания контейнера (Created) и текущее состояние (State).
8. PID процесса внутри хост-системы (State.Pid).
9. Ограничения ресурсов (HostConfig): Memory, CpuShares, RestartPolicy.
10. Метки (Labels) образа и контейнера.

Пример команды для извлечения отдельного поля:

```
docker inspect --format '{{.NetworkSettings.IPAddress}}' <контейнер>
docker inspect <контейнер> | python3 -m json.tool | grep -A3 '"IPAddress"'
```

5.2. Исследование файловой системы контейнера

Подключитесь к контейнеру вашего варианта:

```
docker exec -it <контейнер> sh
```

Внутри контейнера выполните:

```
cat /etc/os-release
ps aux
ls -la /
df -h
```

Зафиксируйте результаты. Выйдите из контейнера: `exit`

5.3. Исследование образа

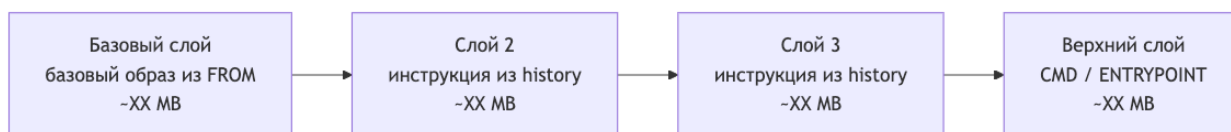
```
docker inspect <контейнер> --format '{{.Config.Image}}'
docker image inspect <образ>
docker image history <образ>
```

В отчёте зафиксируйте:

- Полное имя образа и его digest (sha256).
- Количество слоёв в образе и их суммарный размер.
- Команды, которые выполнялись при сборке каждого слоя (из history).
- Дату сборки образа.

Диаграмма 2 — слоистая структура образа

Постройте диаграмму, визуализирующую слои образа контейнера вашего варианта. Используйте вывод `docker image history` как единственный источник данных — **замените все placeholder-значения на реальные**. Каждый узел — один слой; направление — от базового к верхнему.



Добавьте или уберите узлы в соответствии с реальным количеством слоёв. Укажите реальные размеры (поле SIZE из `docker image history`).

Вопросы к диаграмме:

- Какой слой самый «тяжёлый»? Почему именно он занимает больше всего места?
- Какие слои будут переиспользованы из кэша при повторной сборке, если изменить только исходный код приложения?
- Что произойдёт с кэшем, если поменять местами инструкции `COPY requirements.txt` и `RUN pip install`?

6. Задание 3: Модификация сервиса vote

6.1. Изучение оригинального Dockerfile

Перейдите в каталог сервиса vote:

```
cd vote/
cat Dockerfile
ls -la
```

В отчёте:

- Какой базовый образ используется в оригинальном Dockerfile?
- Какие команды выполняются при сборке? В каком порядке?
- Какой порт экспонирует контейнер?
- Как запускается приложение (CMD/ENTRYPOINT)?

6.2. Модификация Dockerfile

Создайте новый Dockerfile с именем `Dockerfile.custom`. Используйте в качестве базового образа `dockersamples/examplevotingapp_vote` и добавьте страницу с `iframe`.

Не изменяйте оригинальный Dockerfile — работайте с копией Dockerfile.custom.

Пример структуры `Dockerfile.custom`:

```
FROM dockersamples/examplevotingapp_vote

COPY custom_index.html /app/templates/index.html

LABEL maintainer="student@university.ru"
LABEL lab.variant="<ваш вариант>"
```

6.3. Создание модифицированной страницы

Создайте файл `custom_index.html` на основе оригинального шаблона (`vote/templates/index.html`). Требования к странице:

- Страница должна сохранять оригинальную форму голосования.
- `Iframe` должен быть размещён ниже формы голосования, иметь ширину не менее 80% и высоту не менее 400px.
- Над `iframe` добавьте заголовок с описанием источника.
- Страница должна содержать ваши ФИО и вариант в комментарии HTML.

Пример фрагмента для вставки:

```
<h3>Дополнительный материал по теме</h3>
<iframe src="<URL вашего варианта>"
        width="80%" height="400px"
        frameborder="0">
  <p>Ваш браузер не поддерживает iframe.</p>
</iframe>
```

6.4. Сборка и запуск модифицированного образа

```
docker build -f Dockerfile.custom -t vote-custom:v1 .
docker images | grep vote-custom
docker run -d -p 5002:80 --name vote-test vote-custom:v1
docker ps | grep vote-test
```

Откройте в браузере и убедитесь, что `iframe` отображается.

```
docker stop vote-test && docker rm vote-test
```

Для запуска через Compose — замените `build`-секцию сервиса `vote` в `docker-compose.yml`, указав `dockerfile: Dockerfile.custom`, и пересоберите:

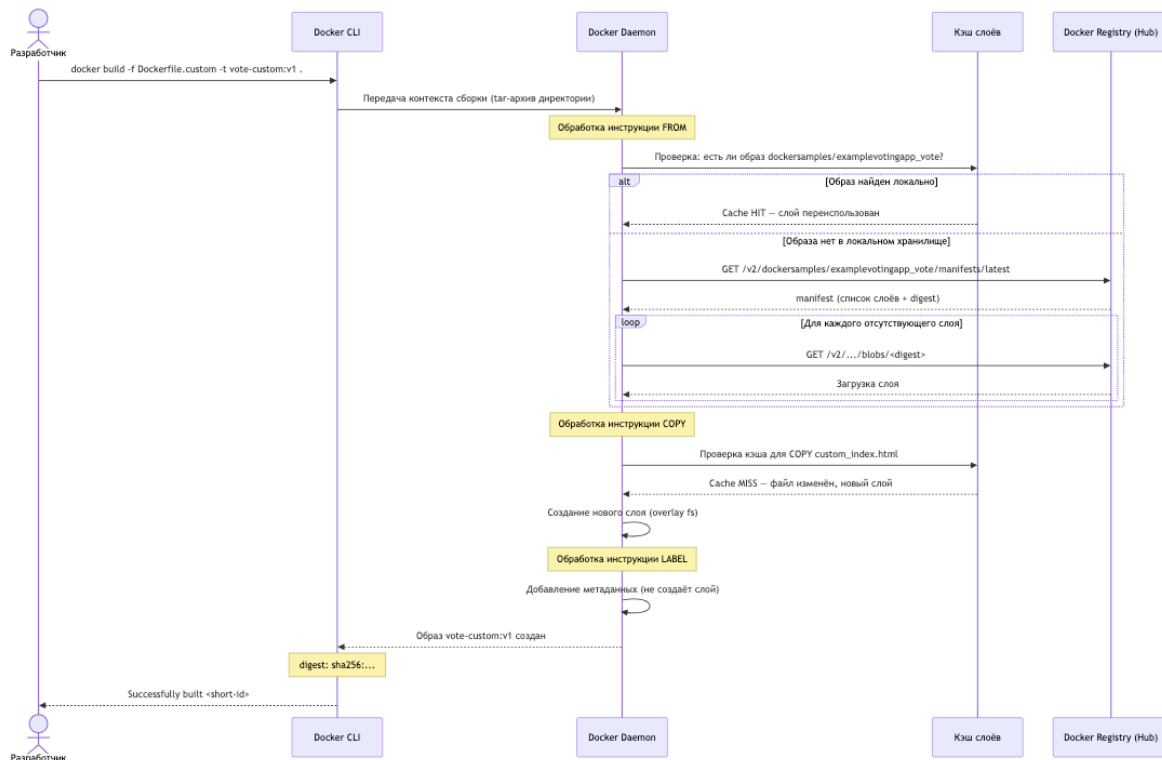
```
docker compose up -d --build vote
```

В отчёте приведите:

- Полный текст `Dockerfile.custom`.
- Полный текст `custom_index.html`.
- Скриншот собранного образа (`docker images`) и запущенного контейнера.
- Скриншот страницы с отображённым `iframe`.

Диаграмма 3 — процесс сборки образа (docker build)

Постройте диаграмму последовательности (sequence diagram), описывающую процесс `docker build` от запуска команды сборки до появления готового образа. Диаграмма должна отражать взаимодействие: разработчик → Docker CLI → Docker Daemon → кэш слоёв → Docker Registry.



Вопросы к диаграмме:

- На каком шаге происходит проверка кэша? Какие инструкции `Dockerfile` инвалидируют кэш всех последующих слоёв?
- Что изменится в диаграмме при повторном запуске `docker build` без изменений в файлах?
- Что происходит с контекстом сборки (tar-архивом)? Почему `.dockerignore` важен для больших проектов?

7. Задание 4: Работа с Git — ветвление и история изменений

Это задание направлено на практическое применение материала лекции 4 (контроль версий и Git). Все действия выполняются в локальном репозитории `example-voting-app`.

7.1. Изучение истории репозитория

```
git log --oneline --graph --all
```

```
git log --stat --follow -- vote/Dockerfile
git log --format="%an" | sort | uniq -c | sort -rn
```

В отчёте:

- Как давно был создан репозиторий? Сколько всего коммитов?
- Перечислите 5 наиболее активных авторов.
- Как изменялся Dockerfile сервиса vote? Сколько раз он правился?
- Найдите коммит, в котором впервые появился docker-compose.yml:

```
git log --diff-filter=A -- docker-compose.yml
```

7.2. Создание ветки и фиксация изменений

```
git checkout -b feature/lab1-variant-<номер_варианта>
git add vote/Dockerfile.custom vote/custom_index.html
git status
git commit -m "feat(vote): add custom template with iframe for variant <N>"
git show HEAD
git log --oneline -5
```

7.3. Работа с тегами и сравнение версий

```
git tag -a v0.1-lab-variant<N> -m "Lab 1 submission, variant <N>"
git tag -l
git diff main -- vote/Dockerfile.custom vote/Dockerfile
```

Сохраните вывод `git diff` в отчёт.

7.4. Имитация командной работы: cherry-pick

Создайте вторую ветку от main:

```
git checkout main
git checkout -b hotfix/readme-update
echo "\n## Variant <N> notes\nCustom vote page added." >> README.md
git add README.md
git commit -m "docs: add variant notes to README"
```

Вернитесь в свою ветку и перенесите изменения из hotfix:

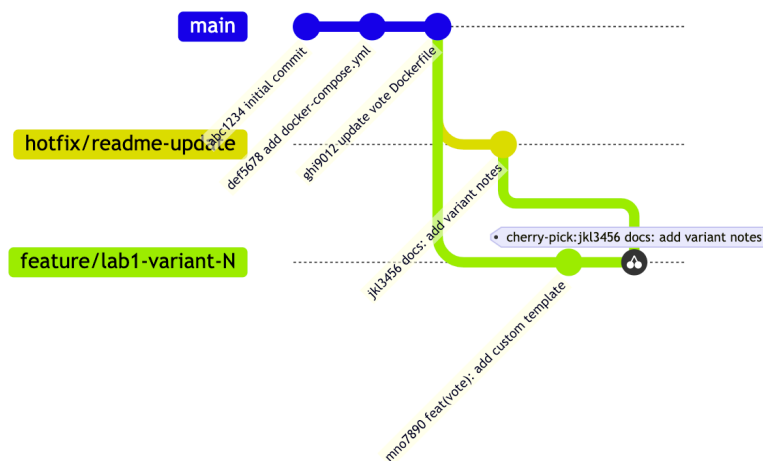
```
git checkout feature/lab1-variant-<N>
git cherry-pick hotfix/readme-update
git log --oneline -5
```

В отчёте: что такое cherry-pick и в каких ситуациях он используется? Чем отличается от merge?

Диаграмма 4 — граф веток Git

Постройте диаграмму, отображающую состояние репозитория после выполнения всех операций задания 4. Диаграмма должна показывать ветки main, feature/lab1-variant-<N>, hotfix/readme-update и операцию cherry-pick.

Замените идентификаторы коммитов на реальные сокращённые хэши из вашего репозитория (из вывода `git log --oneline`):



Вопросы к диаграмме:

- Чем cherry-pick отличается от merge визуально на графе веток?
- Что произойдёт с графом, если выполнить `git rebase main` на ветке `feature/...` вместо cherry-pick?
- В каком сценарии командной разработки использование `merge --no-ff` предпочтительнее `rebase`?

8. Задание 5: Исследование и оптимизация образа

8.1. Анализ размера образов

```
docker images | grep -E 'vote|result|worker|redis|postgres'
docker image history --no-trunc vote-custom:v1
docker inspect vote-custom:v1 --format '{{json .RootFS.Layers}}' | python3 -m json.tool
```

Запишите в отчёт: сколько слоёв в образе? Какой слой занимает больше всего места?

8.2. Предложение оптимизации

На основе анализа предложите и реализуйте конкретный способ уменьшения размера образа. Допустимые подходы (выберите один, обоснуйте выбор):

- Использование более компактного базового образа (например, переход на alpine-вариант).
- Объединение нескольких команд RUN в одну для сокращения числа слоёв.
- Применение многоэтапной (multi-stage) сборки.
- Добавление `.dockerignore` и исключение ненужных файлов из контекста сборки.

```
docker build -f Dockerfile.custom -t vote-custom:v1 .
docker build -f Dockerfile.optimized -t vote-custom:v2-opt .
docker images | grep vote-custom
```

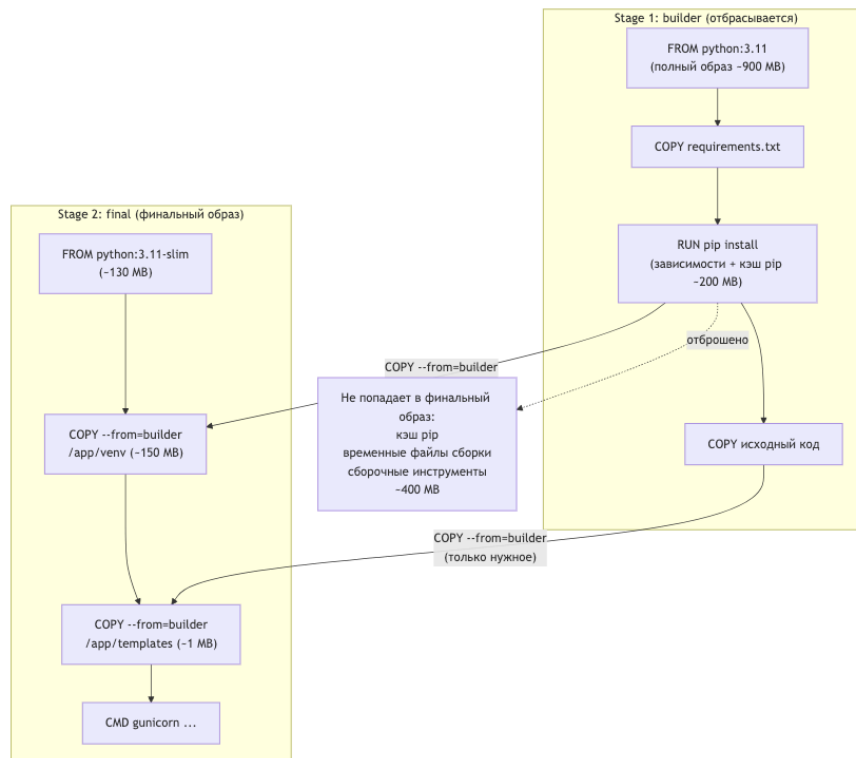
В отчёте:

- Опишите выбранный способ оптимизации и принцип его работы.
- Приведите полный текст `Dockerfile.optimized`.
- Сравните размеры образов `v1` и `v2-opt`. Какой выигрыш в размере (в МБ и процентах)?
- Есть ли недостатки у выбранного подхода?

Диаграмма 5 — сравнение стратегий сборки

Постройте диаграмму, наглядно показывающую разницу между исходным подходом и выбранным способом оптимизации. Если вы использовали multi-stage сборку — покажите, что копируется из builder-стадии и что отбрасывается. Если другой подход — адаптируйте диаграмму под него.

Пример для multi-stage:



Замените цифры на реальные размеры из вашего эксперимента.

Вопросы к диаграмме:

- Какова итоговая экономия в вашем эксперименте (МБ, %)?
- Есть ли у выбранного подхода недостатки (время сборки, сложность поддержки)?
- Когда multi-stage сборка неприменима или нецелесообразна?

8.3. Проверка работоспособности оптимизированного образа

```
docker run -d -p 5003:80 --name vote-opt vote-custom:v2-opt
```

Откройте <http://localhost:5003> и проверьте наличие iframe.

```
docker stop vote-opt && docker rm vote-opt
```

9. Теоретические вопросы к лабораторной работе

Ответы на вопросы включаются в письменный отчёт. Каждый ответ должен быть развёрнутым (не менее 5–7 предложений), с примерами из выполненной работы.

Блок А: Docker (лекция 4, практическая часть)

1. Что такое слой (layer) образа Docker? Почему Docker использует слоистую файловую систему? Назовите технологии Union FS, которые может использовать Docker. Как это влияет на эффективность хранения и передачи образов?
2. Чем образ (image) отличается от контейнера (container)? Объясните аналогию «образ — это класс, контейнер — это экземпляр». В каком состоянии может находиться контейнер?
3. Что такое digest образа (sha256)? Где вы его видели в ходе работы? Почему нельзя использовать только тег для обеспечения воспроизводимости сборки?
4. Что такое Docker API? Как Docker CLI взаимодействует с Docker daemon? Приведите пример прямого обращения к Docker API через `curl` или Python SDK.
5. Что такое Docker Registry? Чем отличается Docker Hub от приватного реестра? Каков процесс команды `docker pull`: откуда и как именно скачивается образ? Что происходит при команде `docker push`?

Блок Б: Git и DevOps (лекции 1–4)

6. Что такое DevOps? Перечислите его ключевые принципы согласно модели CALMS (Culture, Automation, Lean, Measurement, Sharing). Приведите пример каждого принципа применительно к проекту `example-voting-app`.
7. Опишите жизненный цикл разработки ПО (SDLC). Какие стадии вы наблюдаете в репозитории `example-voting-app` (в истории коммитов, CI-файлах, тегах)?
8. Что такое система управления задачами? Какие инструменты относятся к этой категории? Как организация задач связана с ветками в Git (Git Flow, Trunk-based development)?
9. Объясните разницу между `merge` и `rebase`. Когда предпочтительнее использовать каждый из подходов? Что произошло при выполнении `cherry-pick` в задании 4.4?
10. Что произойдёт при выполнении `git pull --rebase` вместо обычного `git pull`? Почему команды `git reset --hard` и `git push --force` считаются опасными в командной работе?

Блок В: Архитектура и инфраструктура

6. Опишите архитектуру `example-voting-app`: какие сервисы входят в состав приложения, на каком языке написаны, какую роль выполняют? Соотнесите свой ответ с **диаграммой 1** из задания 1.
7. Что такое Docker Compose? В чём разница между `docker-compose.yml` и `Dockerfile`? Для каких сред (разработка, staging, production) Compose является подходящим инструментом, а для каких — нет?

10. Требования к отчёту

Отчёт оформляется в формате PDF или DOCX и должен содержать следующие разделы:

1. **Титульный лист:** дисциплина, тема работы, ФИО студента, группа, вариант, дата.
2. **Задание 1:** скриншоты `docker ps` / `docker compose ps`, **диаграмма 1** (архитектура), ответы на вопросы.
3. **Задание 2:** вывод `docker inspect` (основные поля), **диаграмма 2** (слои образа), вывод `docker image history`.
4. **Задание 3:** текст `Dockerfile.custom`, текст `custom_index.html`, **диаграмма 3** (процесс сборки), скриншоты.
5. **Задание 4:** скриншоты `git log`, **диаграмма 4** (граф веток), вывод `git diff`, ответ про `cherry-pick`.

6. **Задание 5:** сравнительная таблица размеров образов, `Dockerfile.optimized`, **диаграмма 5** (сравнение стратегий).
7. **Теоретические вопросы** (блоки А, Б, В).
8. **Вывод:** что было изучено, какие трудности возникли, что было наиболее интересным.

Отчёты, содержащие ответы, очевидно сгенерированные нейросетью без понимания (отсутствие ссылок на конкретные команды/скриншоты из работы, шаблонные фразы, диаграммы с placeholder-данными вместо реальных), **будут возвращены на доработку.**

12. Рекомендации и типичные ошибки

Что проверять, если что-то не работает

- Контейнер не запускается — проверьте `docker compose logs <сервис>` на наличие ошибок.
- `iframe` не отображается — сайт может блокировать встраивание через X-Frame-Options; попробуйте другой URL и зафиксируйте причину.
- `docker exec` возвращает ошибку — возможно, в образе нет `sh` или `bash`; используйте `docker exec <контейнер> ls /`.
- `git push` требует удалённый репозиторий — для локальной работы `git push` не нужен; все задания выполняются в локальном репозитории.
- Ошибка при сборке образа — проверьте, что вы находитесь в каталоге `vote/` и файл `Dockerfile.custom` существует.
- Mermaid не рендерится — используйте mermaid.live, VS Code с расширением Markdown Preview Mermaid, GitLab или GitHub.

Типичные ошибки в отчёте

- Копирование вывода команд без анализа — объясняйте каждый результат.
- Отсутствие скриншотов — без визуального подтверждения работа не засчитывается.
- Диаграммы с шаблонными данными вместо реальных — замените все `<placeholder>`-значения на данные из своего репозитория.
- Ответы на теоретические вопросы без привязки к практике — ссылайтесь на конкретные наблюдения из заданий.
- Нарушение формата коммит-сообщения — используйте Conventional Commits (`feat:`, `fix:`, `docs:` и т.д.).

13. Полезные ресурсы

- Документация Docker: <https://docs.docker.com/>
- Справочник команд Docker CLI: <https://docs.docker.com/engine/reference/commandline/cli/>
- Репозиторий example-voting-app: <https://github.com/dockersamples/example-voting-app>
- Git Reference: <https://git-scm.com/doc>
- Conventional Commits: <https://www.conventionalcommits.org/ru/v1.0.0/>
- The Twelve-Factor App: <https://12factor.net/>
- Docker multi-stage builds: <https://docs.docker.com/develop/develop-images/multistage-build/>
- Mermaid — онлайн-редактор диаграмм: <https://mermaid.live/>
- Mermaid — документация: <https://mermaid.js.org/intro/>

Шкала оценивания и критерии оценки:

Количество лабораторных работ в каждом модуле — 2. Количество баллов за лабораторную работу может варьироваться таким образом, чтобы обеспечить суммарное количество баллов за все лабораторные работы модуля в следующих пределах:

- минимальное количество баллов за лабораторные работы 1 модуля — 12
- максимальное количество баллов за лабораторные работы 1 модуля — 19
- минимальное количество баллов за лабораторные работы 2 модуля — 12
- максимальное количество баллов за лабораторные работы 2 модуля — 19

Оценка	Минимальное количество баллов	Максимальное количество баллов	Критерий
«5» (отлично)	90%	100%	Выполнены все задания лабораторной работы, обучающийся четко и без ошибок ответил на все основные и дополнительные контрольные вопросы
«4» (хорошо)	75%	90%	Выполнены все задания лабораторной работы; обучающийся без ошибок ответил на все основные контрольные вопросы, ответил на дополнительные контрольные вопросы с замечаниями
«3» (удовлетворительно)	60%	75%	Выполнены все задания лабораторной работы с замечаниями; обучающийся ответил на все основные контрольные вопросы с замечаниями
«2» (не удовлетворительно)	0%	0%	Обучающийся не выполнил или выполнил неправильно задания лабораторной работы; обучающийся ответил на контрольные вопросы с ошибками или не ответил на контрольные вопросы

Итого за семестр: минимальное количество — 24 баллов, максимальное количество — 38 балла

РУБЕЖНЫЙ КОНТРОЛЬ

Рубежный контроль проходит в форме письменной работы, состоящей из 2 теоретических вопросов и 1 задачи. Правильный и полный ответ на теоретические вопросы оценивается в 3 балла, правильное решение задачи — в 4 балла. Основанием для снижения баллов является неполнота ответа, неверный ответ в задаче при условии правильного хода решения.

Пример задания рубежной контрольной работы:

1. Как DevOps соотносится с Agile-подходом? Чем отличаются методы DevOps от классических Agile-практик разработки ПО?
2. Поясните концепцию «Инфраструктура как код» (IaC). Какие преимущества даёт использование IaC?
3. Kubernetes: напишите YAML-манифест Secret (apiVersion: v1, kind: Secret) с ключом token и значением s3cr3t (строка закодирована в Base64).

СОБЕСЕДОВАНИЕ ПО ЭКЗАМЕНАЦИОННЫМ БИЛЕТАМ

Промежуточный контроль по дисциплине осуществляется в форме экзамена, при этом проводится оценка компетенций, сформированных по дисциплине. Студенты отвечают на вопросы, указанные в экзаменационном билете. Каждый билет содержит теоретические вопросы.

Примерный перечень вопросов:

4. Что такое DevOps? Каковы её основные цели и ценности?
5. Как DevOps соотносится с Agile-подходом? Чем отличаются методы DevOps от классических Agile-практик разработки ПО?
6. Что такое непрерывная интеграция (CI) и непрерывная доставка (CD)? Зачем нужен подход CI/CD, и какие шаги включает CI/CD-пайплайн?
7. Перечислите этапы жизненного цикла разработки ПО (SDLC) и опишите их основные особенности.
8. Назовите известные модели и методологии разработки ПО и сравните их с DevOps-подходом.
9. Как организуется командная работа в DevOps-команде? Укажите основные роли, принципы взаимодействия и системы управления задачами.
10. Что такое система контроля версий (VCS)? Приведите примеры и опишите основные операции.
11. Что такое Docker? Какие задачи решает Docker и какие основные компоненты в его архитектуре?
12. Чем контейнеризация отличается от традиционной виртуализации? Какие преимущества обеспечивают контейнеры?
13. Что такое Docker-образ и Docker-контейнер? Как образ хранит инструкции создания контейнера, и в чём разница между ними?
14. Объясните назначение Dockerfile. Какие основные инструкции Dockerfile вы знаете и как они влияют на формирование образа.
15. Как работает многослойная файловая система Docker-образов? Почему каждая команда Dockerfile создаёт новый слой и как это влияет на обновление образов и экономию места?
16. Что такое тома (volumes) в Docker? Зачем нужны тома и как они используются для хранения данных за пределами жизненного цикла контейнера?
17. Какие сетевые драйверы доступны в Docker и как они работают?

18. Что такое Docker Compose? Как с его помощью описывать и запускать мультитемные приложения?
19. Опишите основные принципы работы Kubernetes как системы оркестрации контейнеров. Какие задачи Kubernetes решает?
20. Назовите и опишите ключевые объекты Kubernetes: Pod, ReplicaSet, Deployment, Service, Namespace, ConfigMap и Secret. Для каждого укажите его роль в управлении контейнерным приложением.
21. Как в Kubernetes достигается масштабирование и обновление приложений (например, Rolling Update)? Что такое ReplicaSet и как он связан с Deployment?
22. Что такое Helm и Helm Charts в контексте Kubernetes? Как Helm упрощает установку и обновление приложений в кластере?
23. Поясните концепцию «Инфраструктура как код» (IaC). Какие преимущества даёт использование IaC?
24. Что такое Terraform, как он применяется для создания инфраструктуры? Чем Terraform отличается от других инструментов (Ansible, Chef) для управления окружением?
25. Какие критерии выбора инструментов автоматизации и оркестрации следует учитывать? Приведите примеры распространённых CI/CD-систем и инструментов автоматизации.
26. Что такое мониторинг и наблюдаемость (observability) системы? Назовите «три кита» наблюдаемости (логи, метрики, трассировки) и поясните их назначение.
27. Какие инструменты и технологии используются для мониторинга и визуализации? Что такое Prometheus и Grafana?
28. Перечислите важнейшие метрики и показатели состояния сервисов (SLA, SLO, SLI, время безотказной работы, время восстановления) и объясните их смысл. Что такое Service Level Objective (SLO) и Service Level Agreement (SLA)?
29. Что такое DevSecOps? Как обеспечивается безопасность на разных этапах разработки и доставки ПО (сканирование уязвимостей, управление секретами, RBAC)?
30. Какие метрики процессов разработки и поставки ПО вы знаете и зачем они нужны?
31. Что такое управление изменениями и почему важно контролировать изменения в инфраструктуре и коде? Каково значение коммуникации и ролей при управлении изменениями?
32. Опишите основы отладки и анализа логов программной системы. Какие существуют подходы к логированию (централизованная система, levels) и инструменты для анализа логов?
33. Перечислите практики обеспечения надёжности систем (SRE): что такое инцидент, как проводится пост-мортем, что такое «смещение влево» (shift-left) в контексте DevOps/SRE.
34. Git: напишите последовательность команд для создания новой ветки, фиксации изменений и отправки изменений в удалённый репозиторий.
35. Git: напишите команду для слияния ветки feature в основную ветку (merge) и опишите, как разрешать конфликты.
36. Docker: составьте Dockerfile для простого веб-приложения (например, Nginx или Python Flask), включающего инструкции FROM, RUN, COPY, CMD.
37. Docker: напишите команду docker run для запуска контейнера, проброса порта (флаг -p) и подключения тома (флаг -v).
38. Docker Compose: составьте docker-compose.yml для приложения из двух сервисов (например, веб-сервер + база данных), задав порты, сети и тома.
39. Docker: напишите команды для создания пользовательской сети (docker network create) и запуска двух контейнеров в этой сети.
40. Kubernetes: составьте YAML-манифест Deployment (apiVersion: apps/v1) для приложения на базе образа nginx с 3 репликами.

41. Kubernetes: составьте YAML-манифест Service (тип LoadBalancer или NodePort) для доступа к Deployment из предыдущего задания.
42. Kubernetes: напишите YAML-манифест CronJob (с apiVersion: batch/v1beta1), который ежедневно запускает контейнер busybox с командой date.
43. Helm: укажите команду helm install для установки приложения (например, Nginx) из официального репозитория (укажите название релиза и имя чарта).
44. Helm: приведите пример файла values.yaml для Helm Chart, в котором указываются настройки образа и количество реплик приложения.
45. Terraform: составьте минимальную конфигурацию Terraform (файл main.tf) для создания виртуальной машины (например, AWS EC2 или Azure VM), указав провайдера, имя ресурса и базовые параметры.
46. Terraform: составьте конфигурацию Terraform для создания S3-бакета (или аналогичного ресурса облачного хранилища).
47. Terraform: добавьте в конфигурацию (variables.tf) объявление переменной instance_type и используйте её в настройке виртуальной машины.
48. Ansible: напишите простейший плейбук (YAML) для установки и запуска сервиса Nginx на удалённом сервере (используйте модули apt или yum и service).
49. Ansible: напишите задачу (task) с модулем copy, которая копирует файл config.yml из локальной директории в /etc/myapp/config.yml на целевом хосте.
50. CI/CD (Jenkins): составьте пример Jenkinsfile с этапами Build, Test и Deploy, где каждый этап содержит одну команду (например, сборку, запуск тестов и развёртывание).
51. CI/CD (GitLab CI): составьте пример файла .gitlab-ci.yml с одним job test, который выполняет, например, make test.
52. CI/CD (GitHub Actions): составьте пример workflow-файла .github/workflows/ci.yml с job, запускающим сборку и тесты на Ubuntu.
53. Bash: напишите скрипт, который проверяет, запущен ли процесс по его имени или PID, и перезапускает его при необходимости.
54. Bash: приведите команду grep для поиска строки с текстом ERROR в файле системных логов (например, /var/log/syslog).
55. .dockerignore: укажите пример содержимого файла .dockerignore для Node.js-проекта (например, node_modules, .git).
56. Prometheus: составьте правило алерта (Alertmanager rule) в формате YAML, например, чтобы при загрузке процессора выше 80% отправлялся алерт.
57. Prometheus: приведите пример запроса PromQL (например, для вычисления 95-го перцентиля задержки HTTP-запросов или нагрузки на CPU).
58. Kubernetes: напишите YAML-манифест Secret (apiVersion: v1, kind: Secret) с ключом token и значением s3cr3t (строка закодирована в Base64).
59. Kubernetes: напишите YAML-манифест ConfigMap с именем app-config, содержащий ключ config.yml и произвольное значение конфигурации.
60. Kubernetes: составьте YAML-манифест Ingress для маршрутизации HTTP-запросов к Service из задания 8 (укажите базовый пример с хостом и путём).
61. Monitoring: приведите пример настройки scrape_configs в Prometheus (в формате YAML) для сбора метрик с локального узла или сервиса.
62. Логирование: приведите команду journalctl для вывода последних записей лога определённой службой (например, docker.service или nginx.service).

63. Мониторинг: приведите команду `docker stats` или `kubectl top` для просмотра текущей нагрузки (CPU/память) работающих контейнеров.

ПРИМЕР ЭКЗАМЕНАЦИОННОГО БИЛЕТА

Вопрос 1. Что такое DevOps – методология объединения разработки и эксплуатации ПО? Каковы её основные цели и ценности?

Вопрос 2. Перечислите практики обеспечения надёжности систем (SRE): что такое инцидент, как проводится пост-мортем, что такое «смещение влево» (shift-left) в контексте DevOps/SRE.

Вопрос 3. Bash: напишите скрипт, который проверяет, запущен ли процесс по его имени или PID, и перезапускает его при необходимости.

Шкала оценивания и критерии оценки:

Критерии оценки	Минимальное количество баллов	Максимальное количество баллов
Уровень усвоения материала, предусмотренного программой	3	5
Умение выполнять задания, предусмотренные программой	4,2	7
Уровень знакомства с основной литературой, предусмотренной программой	1,2	2
Уровень знакомства с дополнительной литературой	0,6	1
Уровень раскрытия причинно-следственных связей	0,6	1
Уровень раскрытия междисциплинарных связей	0,6	1
Качество ответа (его общая композиция, логичность, убежденность, общая эрудиция)	0,6	1
Ответы на вопросы: полнота, аргументированность, убежденность, умение использовать ответы на вопросы для более полного раскрытия содержания вопроса	0,6	1
Деловые и волевые качества докладчика: ответственное отношение к работе, стремление к достижению высоких результатов, готовность к дискуссии, контактность	0,6	1
Итого баллов:	12	20

Оценка	Минимальное количество баллов	Максимальное количество баллов
«5» (отлично)	90	100
«4» (хорошо)	74	90
«3» (удовлетворительно)	60	74
«2» (неудовлетворительно)	0	60

Знания, умения и навыки обучающихся при промежуточном контроле в форме экзамена определяются оценками «отлично», «хорошо», «удовлетворительно», «неудовлетворительно».

1. «Отлично» – обучающийся глубоко и прочно усвоил весь программный материал, изчерпывающе, последовательно, грамотно и логически стройно его излагает, не затрудняется с ответом при видоизменении задания, свободно справляется с задачами и практическими заданиями, правильно обосновывает принятые решения, умеет самостоятельно обобщать и излагать материал, не допуская ошибок.

2. «Хорошо» – обучающийся твердо знает программный материал, грамотно и по существу излагает его, не допускает существенных неточностей в ответе на вопрос, может правильно применять теоретические положения и владеет необходимыми умениями и навыками при выполнении практических заданий.

3. «Удовлетворительно» – обучающийся усвоил только основной материал, но не знает отдельных деталей, допускает неточности, недостаточно правильные формулировки, нарушает последовательность в изложении программного материала и испытывает затруднения в выполнении практических заданий.

4. «Неудовлетворительно» – обучающийся не знает значительной части программного материала, допускает существенные ошибки, с большими затруднениями выполняет практические задания, задачи.